

Phase 17: Retrieval Evaluation

Purpose

`17_evaluate_retrieval.py` checks whether the retriever returns the correct hospital document chunks for each Phase 16 test question.

The phase calls the real Phase 7 retriever against the Phase 6 vector index. It does not use a separate mock retrieval algorithm. This keeps the evaluation honest and makes the output useful for tuning chunking, metadata, embeddings, and retrieval routing.

Input Files

File	Purpose
<code>01_data/processed/16_test_set.csv</code>	Phase 16 test cases with questions, expected sources, categories, and safety expectations.
<code>05_vector_store/chroma_db/06_vector_index.sqlite3</code>	Phase 6 vector index searched by the retriever.
<code>04_embeddings/05_create_embeddings.py</code>	Query embedding implementation loaded by the Phase 7 retriever.
<code>04_embeddings/06_store_vector_index.py</code>	Vector-index query interface loaded by the Phase 7 retriever.
<code>06_rag_pipeline/07_retriever.py</code>	Real retrieval implementation under evaluation.

Output Files

All generated files use the `17_` prefix so they line up with `17_evaluate_retrieval.py`.

File	Purpose
<code>01_data/processed/17_retrieval_results.json</code>	Full scored result for every evaluated test case.
<code>01_data/processed/17_retrieval_report.json</code>	Summary metrics: pass rate, source-hit rate, category-hit rate, safety-hit rate, confidence counts, and latency.
<code>01_data/processed/17_retrieval_audit.csv</code>	Spreadsheet-friendly audit table for manual review.
<code>01_data/processed/17_failed_retrieval_queries.json</code>	Runtime failures, such as missing index or invalid question input.

01_data/processed/17_retrieval_misses.json	Test cases where expected evidence was not found in top-k retrieval.
--	--

Plots Generated

Plot	Purpose
01_data/processed/plots/17_retrieval_pass_rate_by_category.png	Shows which categories pass or need retrieval tuning.
01_data/processed/plots/17_retrieval_score_by_test_case.png	Shows top retrieval score per test case and highlights misses.
01_data/processed/plots/17_retrieval_latency_by_test_case.png	Shows retrieval latency for each test question.

Code Section Guide

1. Configuration

`RetrievalEvaluationConfig` defines the project root, Phase 16 test-set path, Phase 6 vector-index path, processed output folder, plots folder, and top_k. The defaults match the project folder structure.

2. Dataclasses

`RetrievalEvaluationRow` stores one scored test case. It records expected sources, retrieved sources, retrieved categories, source-hit status, category-hit status, safety routing status, pass/fail status, confidence, latency, and miss reason.

`RetrievalEvaluationResult` returns every generated file path plus headline metrics.

3. Project Root Resolution

`resolve_project_root()` allows the module and notebook to run from the workspace root, project root, 09_evaluation, or 13_notebooks directory.

4. Loading the Real Retriever

`load_retriever()` imports 06_rag_pipeline/07_retriever.py. The evaluator then uses the retriever's own `load_dependencies()` and `retrieve()` functions.

5. Reading Phase 16 Test Cases

`read_test_set()` reads `16_test_set.csv` and validates required fields: `test_id`, `question`, `category`, `safety_class`, `expected_sources`, `expected_mode`, and `expected_guardrail_action`.

6. Source, Category, and Safety Matching

The evaluator records three checks:

- `source_hit`: expected source name appears in the retrieved evidence.
- `category_hit`: retrieved evidence matches the expected helpdesk category.
- `safety_hit`: emergency or unsafe medical questions receive the expected retrieval-time safety label.

The pass rule is transparent: safety-guardrail cases pass when safety routing is detected; normal cases pass when either the expected source or expected category appears in the top-k evidence.

7. Reports and Plots

The phase writes full JSON results, a compact CSV audit, misses, runtime failures, a JSON report, and three diagnostic plots. Misses are not hidden: they are first-class artifacts for retrieval tuning.

8. CLI Entry Point

Run from the project root:

```
python 09_evaluation/17_evaluate_retrieval.py
```

or from the workspace root:

```
python hospital_patient_helpdesk_chatbot/09_evaluation/17_evaluate_retrieval.py
```

Notebook and Python File Alignment

The notebook imports `09_evaluation/17_evaluate_retrieval.py` and calls `evaluate_retrieval()` directly. That means the notebook and Python file use the same logic, the same paths, and the same output names.

Difference Between `.ipynb` and `.py`

File	Role	Best Use
13_notebooks/17_evaluate_retrieval.ipynb	Interactive walkthrough with previews, validation cells, and plot display.	Learning, inspection, and manual review of retrieval quality.
09_evaluation/17_evaluate_retrieval.py	Reusable evaluation module and CLI workflow.	Automation, repeatable pipeline evaluation, and later CI checks.

The notebook is explanatory. The Python file is operational. They stay aligned because the notebook imports the Python file instead of duplicating the retrieval-evaluation algorithm.

Safety Notes

- Retrieval evaluation checks evidence selection only; it does not generate final medical answers.
- Emergency and unsafe-medical-advice questions are evaluated for routing, not diagnosis or dosage content.
- Missed retrieval cases should be reviewed before using the chatbot with real hospital users.